МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

УНИВЕРСИТЕТ ИТМО

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 3

Технология MapReduce

Сравнение MapReduce и DuckDB при анализе ИТ-вакансий

Вариант 1: программист, разработчик на 1С

Выполнил:
Карташов Игорь Олегович
Группа J3312

Санкт-Петербург
2026

1. Стенд и исходные данные

Я сравнил собственный MapReduce и DuckDB на вакансиях разработчиков 1С: посчитал навыки, пары и тройки, проверил совпадение результатов, измерил время и память. Затем сравнил зарплаты и три региона.

Параметр	Значение
Компьютер	Windows 11, Ryzen 7 7840H, 8 ядер / 16 потоков, 32 ГБ ОЗУ
Программы	RustFS 1.0.3; Python 3.12.14; DuckDB 2.0.0; boto3 1.43.118
Хранилище	Стенд работы 2: четыре папки, EC 2+2; S3 127.0.0.1:9000; бакет hh-lake

Основной источник — it_vacancies_2024.csv преподавателя: **229 388 вакансий, 847 619 384 байта**. В нём есть должность, роль, регион, навыки и зарплата. Все папки RustFS находятся на одном физическом диске E:.

Из архива на 10 000 JSON я случайно выбрал 1 000 файлов с seed=42 и загрузил их как JSONL. В этой выборке только 74 вакансии варианта, поэтому основной расчёт выполнен по полному CSV.

```
.\Run-Lab.ps1 init-storage
.\Run-Lab.ps1 audit
.\Run-Lab.ps1 upload

Сводка загрузки:
hh-lake: HTTP 200
raw/it_vacancies_2024.csv: 847619384 bytes; HTTP 200
raw/hh-sample-1000.jsonl: 5402691 bytes; HTTP 200
```

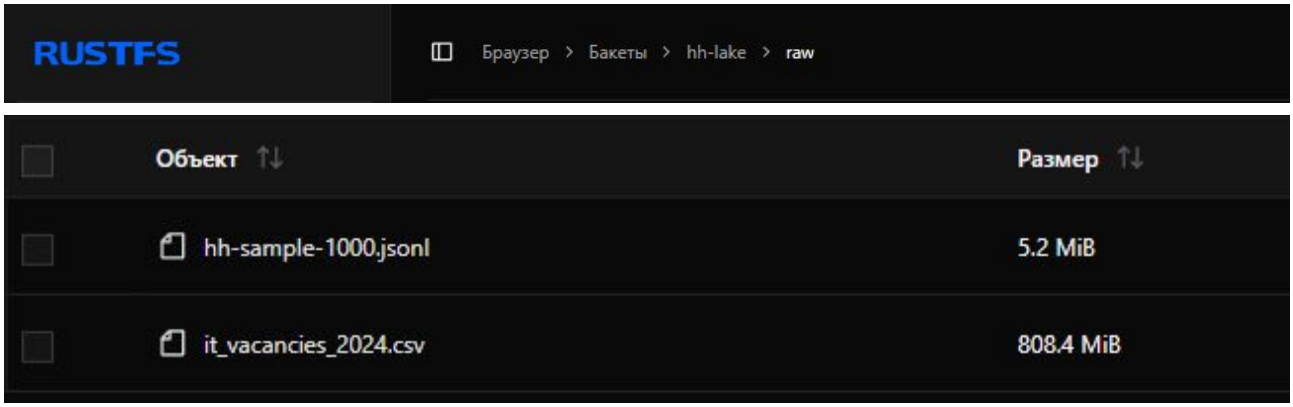


Рисунок 1. Бакет hh-lake: исходный CSV и случайная выборка JSON.

У каждого объекта проверены размер и контрольная сумма. При полном чтении CSV через MapReduce SHA-256 совпал с исходным файлом.

2. Отбор варианта и собственный MapReduce

Я отбирал программистов и разработчиков на 1С по роли и названию вакансии. Исключил Битрикс и Java Developer: упоминание продукта 1С в названии ещё не означает работу на языке 1С.

```
role = 'Программист, разработчик'
AND regexp_matches(lower(name), '1[cc]')
AND regexp_matches(lower(name),
    'программист|разработчик|developer|programmer')
AND NOT regexp_matches(lower(name), 'битрикс|bitrix')
AND NOT regexp_matches(lower(name), 'java[[:space:]-]+developer')
```

Пример названия	Результат
Программист 1С; Разработчик 1С:CRM	Включены при нужной роли
Консультант 1С; Системный аналитик 1С	Исключены
PHP / Программист 1С-Битрикс в веб-студию	Исключена
Java Developer (разработка технологии 1С:Предприятие.Элемент)	Исключена

Показатель	Вакансий
Все строки CSV; повторных source:vacancy_id нет	229 388
Упоминание 1С/1С в названии	31 166
После полного условия варианта	18 164
Из них с непустым списком навыков	11 842

Как устроен расчёт

Сначала get_object открывает поток S3. Я читаю его порциями по 1 МиБ и передаю CSV-парсеру, который сохраняет многострочные поля и символы UTF-8 на границе порций. За полный проход прочитаны 847 619 384 байта за 809 чтений тела ответа.

Строки собираются в батчи по 1 000 записей; четыре процесса-маппера независимо применяют условие варианта. Навыки разделяются по «;», очищаются от пробелов и приводятся к нижнему регистру. Повторы навыка внутри одной вакансии удаляются.

Маппер выдаёт неупорядоченные сочетания размеров 1, 2 и 3. Затем reducer суммирует одинаковые ключи. У вакансии с n навыками получается $C(n,k)=n!/[k!(n-k)!]$ сочетаний. Например, шесть навыков дают 6 одиночных записей, 15 пар и 20 троек.

3. Независимый SQL и проверка частот

DuckDB читает тот же s3://hh-lake/raw/it_vacancies_2024.csv через httpfs. Условие варианта написано отдельно на SQL. После очистки списка навыков я присвоил им позиции и построил сочетания соединением строк одной вакансии:

```
SELECT a.vacancy_key, a.skill, b.skill
FROM skill_positions a JOIN skill_positions b
  ON a.vacancy_key=b.vacancy_key AND a.position<b.position;

SELECT a.vacancy_key, a.skill, b.skill, c.skill
FROM skill_positions a
JOIN skill_positions b ON a.vacancy_key=b.vacancy_key
                      AND a.position<b.position
JOIN skill_positions c ON a.vacancy_key=c.vacancy_key
                      AND b.position<c.position;
```

Размер стека	Выдано сочетаний	Разных ключей	Расхождений
1	64 471	2 178	0
2	214 983	28 430	0
3	632 469	174 582	0

Проверены все ключи и частоты, а также состав выбранных вакансий по SHA-256 отсортированного списка идентификаторов. Оба способа выбрали 18 164 вакансии. Суммы частот совпали с суммой C(n,k) по всем вакансиям.

```
.\Run-Lab.ps1 verify

Сводка проверки:
all_equal: true
selected_vacancies: 18164
different_keys: 0 (k=1, k=2, k=3)
```

Наиболее частые навыки

Навык	Вакансий	Доля от 18 164
1с программирование	9 299	51,2%
1с: предприятие 8	4 762	26,2%
1с: бухгалтерия	3 168	17,4%
1с: управление торговлей	2 829	15,6%
обновление конфигурации 1с	2 457	13,5%

От одиночных навыков к парам число выдач выросло в 3,33 раза, от пар к тройкам — ещё в 2,94 раза. Это результат перебора сочетаний внутри каждой вакансии.

4. Время, память и оптимизация

Каждый подход я запускал трижды отдельным процессом. MapReduce использовал четыре процесса, DuckDB — четыре потока с лимитом памяти 2 ГБ. Порядок менялся между сериями. Время включает запуск, чтение S3, вычисление и запись таблиц; httpfs был установлен заранее.

Память измерял внешним монитором как максимум суммарного RSS клиента и его дочерних процессов. Между опросами задана пауза 5 мс; сам опрос тоже занимает время. Сервер RustFS в эти значения не входит. Кеш ОС между запусками не очищался.

Способ	Попытка	Время, с	Пик RSS, МиБ
MapReduce	1	24,69	394,9
MapReduce	2	23,25	392,0
MapReduce	3	22,89	393,9
MapReduce с оптимизацией	1	21,07	267,7
MapReduce с оптимизацией	2	19,88	267,2
MapReduce с оптимизацией	3	19,15	266,6
DuckDB	1	8,63	1467,4
DuckDB	2	6,49	1467,9
DuckDB	3	6,49	1466,2

Способ	Медиана времени, с	Медиана пиков RSS, МиБ
MapReduce	23,25	393,9
MapReduce с оптимизацией	19,88	267,2
DuckDB	6,49	1467,4

В базовой версии мапперам передавались полные строки, включая описание вакансии, а промежуточные записи накапливались перед reduce. В оптимизированной версии я оставил для передачи пять нужных полей и добавил локальный combiner. Время сократилось на **14,5%**, память — на **32,2%**; результаты не изменились.

DuckDB оказался в 3,06 раза быстрее оптимизированного MapReduce, но использовал в 5,49 раза больше памяти. SQL выполняет разбор и агрегацию в движке; у Python добавляются создание объектов и обмен батчами между процессами.

5. Отказ маппера и результаты в S3

Для проверки отказа я завершил процесс, обрабатывающий второй батч, с кодом 73. Родитель получил BrokenProcessPool, создал новый пул и повторил незафиксированный батч. Уже учтённые батчи повторно в reducer не попадали.

```
.\Run-Lab.ps1 fault
.\Run-Lab.ps1 verify

Сводка проверки отказа:
mapper_process_exit: 73
uncommitted_batches_replayed: 1
committed_batches: 230 / 230
selected_vacancies: 18164
different_keys_after_recovery: 0
```

После восстановления частоты и выбранные идентификаторы полностью совпали с обычным запуском. Отдельные проверки подтвердили чтение UTF-8 и многострочного CSV при границе порции в три байта, удаление повторных навыков и исключение чужих профессий.

Файл результата	Размер, байт	Проверка обратным чтением
skills_1.csv	86 131	Совпал
skills_2.csv	1 951 033	Совпал
skills_3.csv	17 942 846	Совпал

Все три файла записаны отдельно для MapReduce и DuckDB в hh-lake/results/variant-01/. Итого сохранено шесть объектов. Соответствующие файлы двух способов совпадают побайтно.

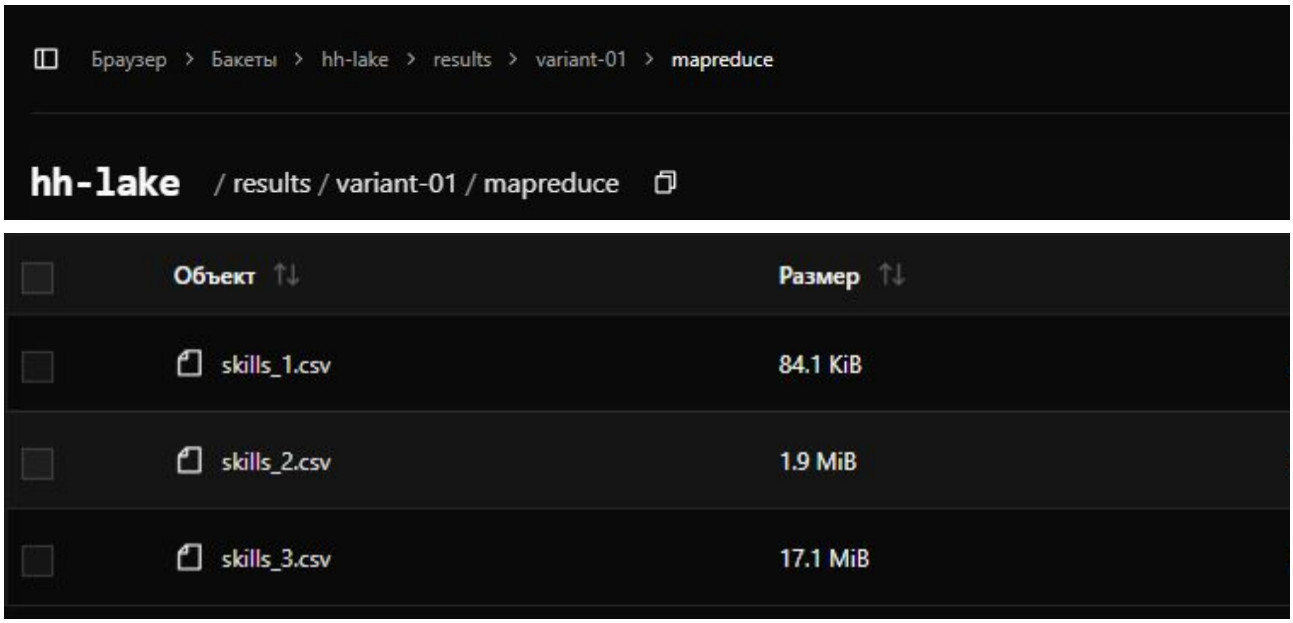


Рисунок 2. Результаты MapReduce для одиночных навыков, пар и троек в S3.

6. Зарплаты: пары навыков

В зарплатное сравнение вошли **7 594 вакансии** с рублёвой суммой на руки (gross=false). При двух границах я брал их среднее, при одной — указанную границу. Медиана сегмента составила **150 000 руб.** Вакансии без такой зарплаты остаются в общих частотах навыков.

Для пары база — более частый отдельный навык по всему сегменту. Сравниваю медиану вакансий с базой M0 и медиану вакансий со всей парой M: Δ=M-M0. В рейтинг включены сочетания хотя бы с 20 зарплатами; n0/n — число зарплатных наблюдений базы и сочетания.

Базовый навык	Добавлен навык	n0 / n	M0, руб.	M, руб.	Разница
1c программирование N0=9 299	java N=45	4 107 / 27	150 000	270 000	+120 000 +80,0%
sql N0=805	java N=41	368 / 25	175 000	270 000	+95 000 +54,3%
git N0=226	java N=27	92 / 21	175 000	270 000	+95 000 +54,3%
erp-системы на базе 1c N0=2 287	работа с базами данных N=100	823 / 28	180 000	100 000	-80 000 -44,4%
1c программирование N0=9 299	ориентация на результат N=33	4 107 / 20	150 000	75 000	-75 000 -50,0%
аналитическое мышление N0=586	алгоритмы N=23	289 / 23	150 000	80 000	-70 000 -46,7%

N0 и N — частоты базы и полного сочетания среди всех вакансий варианта. Из 680 пар с нужным числом зарплат выше показаны три наибольшие положительные и три наибольшие отрицательные разницы.

У пары «1С программирование + Java» медиана выше базы на 120 000 руб., но 18 из 27 зарплатных объявлений размещены одним работодателем. Если исключить его из обеих групп, у оставшихся девяти объявлений пары медиана равна 30 000 руб., у базы — 150 000 руб. Высокий результат здесь связан с составом объявлений; считать его обычной надбавкой за Java нельзя.

У «ERP-системы на базе 1С + работа с базами данных» медиана ниже на 80 000 руб. В этой группе 28 зарплат от 18 работодателей. Без крупнейшего работодателя остаются 22 наблюдения с медианой 137 500 руб.; разрыв с базой сокращается до 42 500 руб. Состав работодателей заметно влияет и на отрицательные результаты.

7. Зарплаты: тройки навыков

Для тройки базой служит самая частая из трёх входящих пар. Частота определяется по всем вакансиям варианта; при равенстве выбирается первая пара по алфавиту. Порог в 20 зарплатных наблюдений и формула разницы те же, что для пар.

Базовая пара	Добавлен навык	n0 / n	M0, руб.	M, руб.	Разница
1с: бухгалтерия + 1с: зарплата и управление персоналом N0=286	1с: торговля N=32	122 / 20	150 000	259 500	+109 500 +73,0%
1с программирование + sql N0=651	java N=37	301 / 23	175 000	270 000	+95 000 +54,3%
1с программирование + git N0=204	java N=26	86 / 20	175 000	270 000	+95 000 +54,3%
1с программирование + erp-системы на базе 1с N0=1 867	работа с базами данных N=82	705 / 27	180 000	100 000	-80 000 -44,4%
1с: комплексная автоматизация + создание конфигурации 1с N0=172	1с: предприятие N=41	80 / 20	175 000	100 000	-75 000 -42,9%
1с: предприятие 8 + erp-системы на базе 1с N0=1 045	обучение и развитие N=41	398 / 27	182 500	115 000	-67 500 -37,0%

Из 1 472 троек с достаточным числом зарплат наибольшая положительная разница — **109 500 руб. (+73%)**: к паре «1с: бухгалтерия + 1с: зарплата и управление персоналом» добавляется «1с: торговля». Медианы базы и тройки — 150 000 и 259 500 руб.

У этой тройки 20 зарплатных наблюдений от шести работодателей, причём 14 объявлений приходятся на одного. После его исключения остаются шесть наблюдений с медианой 250 000 руб. Для общей оценки зарплат такой группы мало.

Наибольшая отрицательная разница — **80 000 руб. (-44,4%)** у добавления «работа с базами данных» к паре «1С программирование + ERP-системы на базе 1С»: 100 000 против 180 000 руб. После исключения крупнейшего работодателя медиана тройки повышается до 125 000 руб., а база остаётся на 180 000 руб.

Разницу медиан нельзя считать прибавкой конкретному работнику: сравниваются разные вакансии, работодатели и требования к опыту.

8. Регионы: число вакансий и зарплаты

Я выбрал Москву, Санкт-Петербург и Екатеринбург — три региона с наибольшим числом вакансий моего варианта. Сравнение включает весь сегмент, а не только объявления с заполненными навыками.

```
SELECT s.area, count(*) AS vacancies,
       count_if(len(s.skills)>0) AS with_skills,
       count(p.salary) AS salary_vacancies, median(p.salary)
FROM selected s JOIN paid p USING(vacancy_key)
GROUP BY s.area ORDER BY vacancies DESC LIMIT 3;
```

Регион	Всего	С навыками	С зарплатой	Медиана, руб.
Москва	5 054	3 301	1 800	200 000
Санкт-Петербург	1 641	1 111	688	175 000
Екатеринбург	862	582	493	150 000

Москва содержит 27,8% всех вакансий варианта. Её медиана выше Екатеринбурга на 50 000 руб. (+33,3%) и Санкт-Петербурга на 25 000 руб. (+14,3%).

Доля объявлений с сопоставимой зарплатой различается: 35,6% в Москве, 41,9% в Санкт-Петербурге и 57,2% в Екатеринбурге. Медианы рассчитаны только по этим объявлениям.

Наиболее частый отдельный навык

Регион	Навык	Вакансий	Доля региона
Москва	1с программирование	2 482	49,1%
Санкт-Петербург	1с программирование	927	56,5%
Екатеринбург	1с программирование	506	58,7%

Во всех трёх регионах лидирует «1С программирование». Его доля выше в Екатеринбурге (58,7%), затем идут Санкт-Петербург (56,5%) и Москва (49,1%). Для долей использовал все вакансии региона, включая объявления без навыков.

9. Регионы: пары и тройки навыков

Для каждого региона я сгруппировал сочетания отдельно. Ниже — самая частая пара и тройка; медиана рассчитана по вакансиям этой комбинации с сопоставимой рублёвой зарплатой.

```
SELECT p.area,c.stack_size,c.skill_1,c.skill_2,c.skill_3,
       count(*) AS vacancies, count(p.salary) AS salary_n,
       median(p.salary) AS median_salary
FROM combinations c JOIN paid p USING(vacancy_key)
GROUP BY p.area,c.stack_size,c.skill_1,c.skill_2,c.skill_3;
```

Регион / k	Наиболее частое сочетание	Частота / доля	Медиана, руб. / n
Москва k=2	1с программирование + 1с: предприятие 8	1 037 / 20,5%	200 000 n=470
Москва k=3	1с программирование + 1с: бухгалтерия + 1с: предприятие 8	352 / 7,0%	210 000 n=163
Санкт-Петербург k=2	1с программирование + 1с: предприятие 8	379 / 23,1%	177 500 n=194
Санкт-Петербург k=3	1с программирование + 1с: предприятие 8 + 1с: управление торговлей	124 / 7,6%	165 000 n=71
Екатеринбург k=2	1с программирование + 1с: предприятие 8	209 / 24,2%	150 000 n=138
Екатеринбург k=3	1с программирование + 1с: бухгалтерия + 1с: предприятие 8	71 / 8,2%	150 000 n=49

Лидирующая пара одинакова: «1С программирование + 1С: Предприятие 8». Она встречается в 20,5% московских, 23,1% петербургских и 24,2% екатеринбургских вакансий. При этом её медианы различаются: 200 000, 177 500 и 150 000 руб. соответственно.

В Москве и Екатеринбурге к лидирующей паре в самой частой тройке добавляется «1С: Бухгалтерия», в Санкт-Петербурге — «1С: Управление торговлей». У петербургской тройки медиана 165 000 руб., что ниже общей медианы города 175 000 руб. Самое частое сочетание не обязательно самое высокооплачиваемое.

Вывод

MapReduce и DuckDB дали одинаковые частоты всех 205 190 разных навыков и сочетаний. Для этой задачи я выбрал бы DuckDB: он быстрее, а его расход памяти укладывается в 32 ГБ стенда. Потоковый MapReduce полезен при более жёстком ограничении памяти. Отказ маппера не изменил результат. При сравнении зарплат нужно учитывать регион и работодателя: у одной из лидирующих пар 18 из 27 зарплатных объявлений размещены одной компанией.

Код и отчёт: github.com/IGK-arch/itmo-mapreduce-1c.

Приложение. Ход работы



Рисунок 3. Свойства исходного CSV: размер, тип и время загрузки.



Рисунок 4. SQL-фильтр варианта 1С в опубликованном коде.

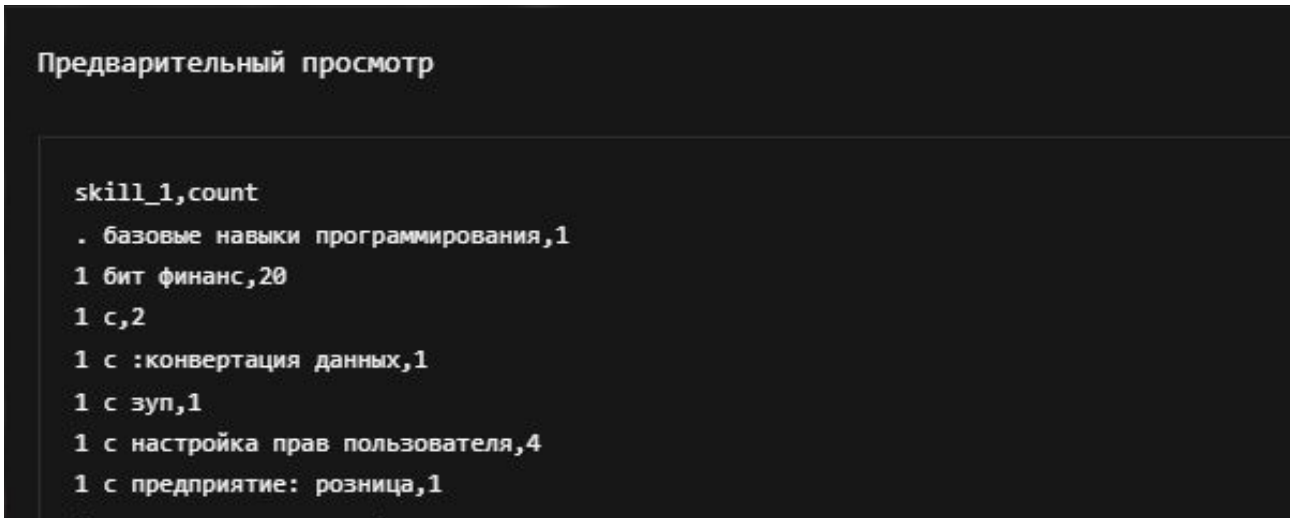


Рисунок 5. Начало CSV с частотами навыков в RustFS.

Приложение. Замеры и проверка

itmo-mapreduce-1c / evidence / benchmark-table.csv

	Method	Attempt	Wall_s	Peak_RSS_MiB	Exit_code
2	baseline	1	24.687	394.9	0
3	baseline	2	23.245	392.0	0
4	baseline	3	22.891	393.9	0
5	optimized	1	21.069	267.7	0
6	optimized	2	19.884	267.2	0
7	optimized	3	19.155	266.6	0
8	duckdb	1	8.635	1467.4	0
9	duckdb	2	6.491	1467.9	0
10	duckdb	3	6.490	1466.2	0

Рисунок 6. Время и пик RSS в девяти запусках.

itmo-mapreduce-1c / evidence / data-checks.txt

```
input: s3://hh-lake/raw/it_vacancies_2024.csv
variant: 1 / Программист, разработчик на 1С
selected_vacancies: 18164
MapReduce / DuckDB: full key and frequency comparison
k=1: total=64471, unique=2178, differences=0
k=2: total=214983, unique=28430, differences=0
k=3: total=632469, unique=174582, differences=0
selected IDs: identical
verified runs: 9 regular + 1 mapper failure
mapper exit code: 73; recovered; committed batches: 230/230
S3 results: 6 objects; full readback: identical
salary groups independently checked: 37
regional groups independently checked: 45
all_checks_passed: true
```

Рисунок 7. Полная сверка результатов и отказ маппера.

Приложение. Запуск расчёта

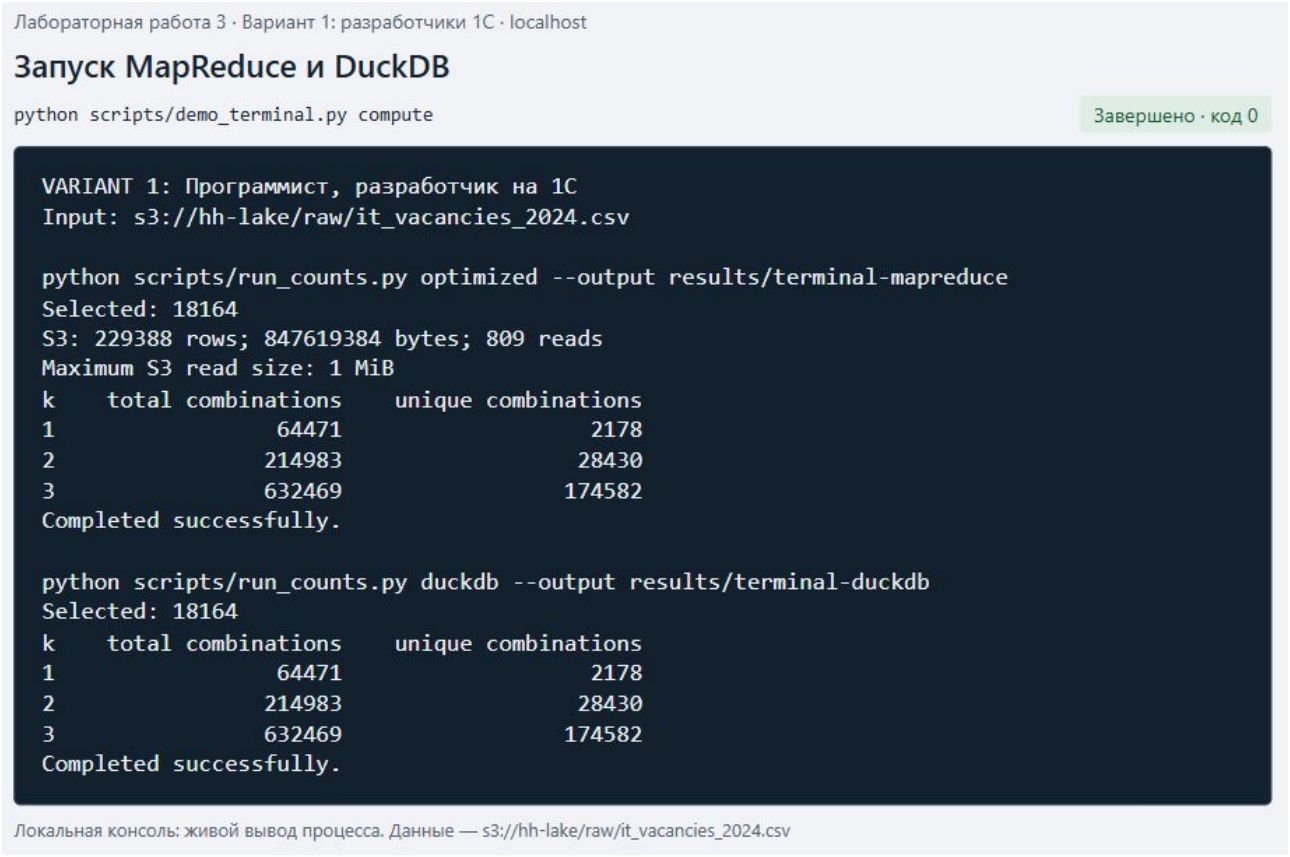


Рисунок 8. Локальная консоль: повторный запуск MapReduce и DuckDB.

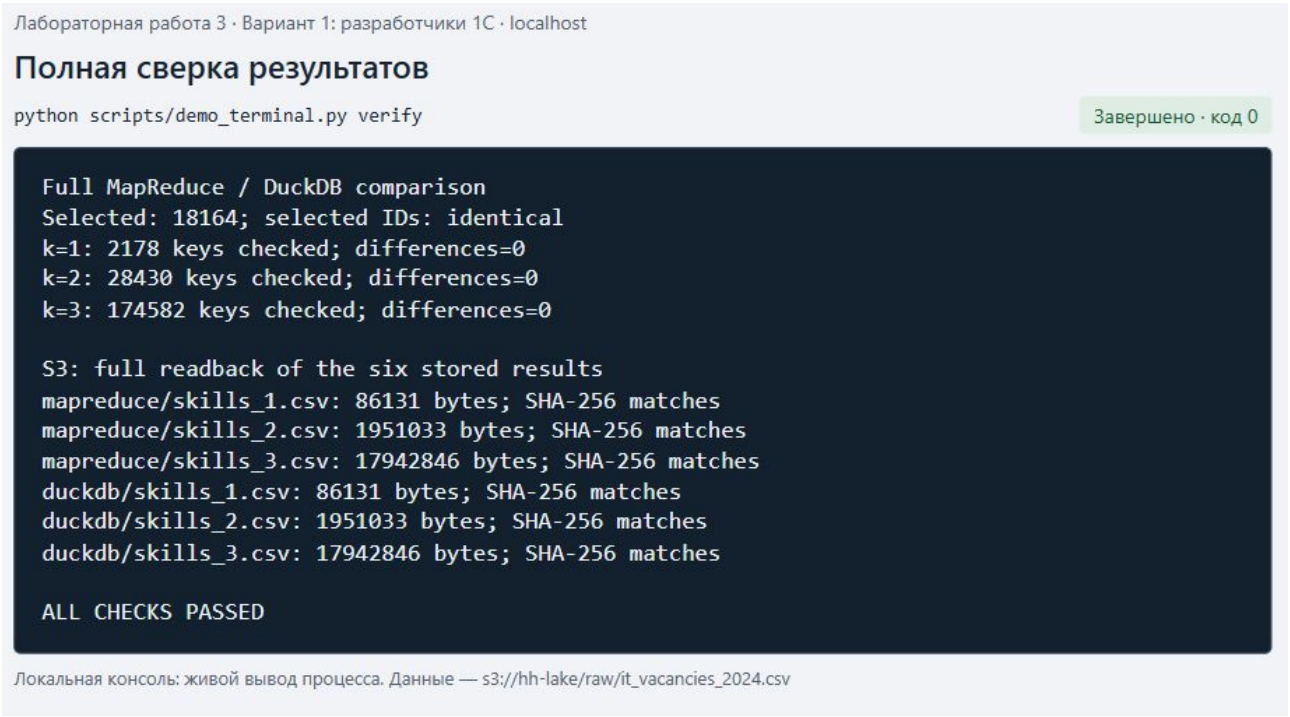


Рисунок 9. Локальная консоль: полная сверка и обратное чтение из S3.